

Tìm kiếm trên đáp án (Binary Search on the Answer)

PHỤ THUỘC A/01 cho heuristic “đảo ngược câu hỏi”; A/04 cho bất biến của tìm kiếm nhị phân; B/06 cho các phép quét dùng trong hàm kiểm tra.

TÓM TẮT

Đây là chương có tỉ lệ “biến bài khó thành bài dễ” cao nhất trong quyển sách, và ý tưởng chỉ gồm một câu: thay vì hỏi “đáp án là bao nhiêu”, hãy hỏi “đáp án có đạt được mức x không” — câu thứ hai thường dễ hơn nhiều bậc. Nếu câu trả lời cho câu thứ hai có tính đơn điệu theo x thì tìm nhị phân trên x cho lời giải với chi phí bằng chi phí kiểm tra nhân log. Chương cũng nói về hai họ hàng: tìm kiếm ba phân cho hàm một cực trị, và các phương pháp tìm kiếm cục bộ khi không có cấu trúc đơn điệu nào. Nếu chỉ nhớ một điều: dấu hiệu nhận biết là đề bài có dạng “nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất” — gặp cụm từ đó thì gần như chắc chắn nên tìm nhị phân trên đáp án.

KIẾN THỨC CẦN CÓ TRƯỚC

- **vị từ đơn điệu** — hàm $P(x)$ nhận giá trị đúng/sai sao cho nếu $P(x)$ đúng thì $P(y)$ đúng với mọi $y > x$ (hoặc mọi $y < x$). Đây là điều kiện sống còn của cả chương.
- **tìm nhị phân trên đáp án** — nhị phân trên miền giá trị của kết quả, mỗi bước gọi một hàm kiểm tra “có đạt được mức này không”.
- **hàm kiểm tra (predicate)** — hàm trả lời câu hỏi quyết định; nó thường là một phép quét tham lam hoặc một quy hoạch động đơn giản.
- **hàm một cực trị (unimodal)** — hàm tăng rồi giảm (hoặc ngược lại), không có cực trị địa phương nào khác — điều kiện để dùng tìm kiếm ba phân.
- **tìm kiếm ba phân** — thu hẹp khoảng bằng cách so hai điểm bên trong; dùng cho hàm một cực trị, tương tự nhị phân nhưng cho bài tối ưu thay vì bài quyết định.
- **tìm kiếm cục bộ** — lớp phương pháp đi từ một lời giải và cải thiện dần bằng các thay đổi nhỏ; không có bảo đảm tối ưu nhưng dùng được khi không còn cấu trúc nào khác.

CÂU HỎI MỞ ĐẦU

- Q1. Vì sao câu hỏi “đáp án có $\geq x$ không” lại thường dễ hơn hẳn câu hỏi “đáp án là bao nhiêu”?
- Q2. Kỹ thuật này đòi tính đơn điệu. Làm sao kiểm tra tính đơn điệu *trước khi* cài, và nó hỏng thế nào nếu bạn nhầm?
- Q3. Tìm nhị phân trên số thực khác gì trên số nguyên, và vì sao “lặp cho tới khi hai đầu bằng nhau” lại là một lỗi?
- Q4. Vì sao tìm kiếm ba phân cần hàm *một cực trị* chứ không chỉ cần “có cực đại”?
- Q5. Khi bài toán không có tính đơn điệu và cũng không có cấu trúc nào khác, còn lại lựa chọn gì?

Trong danh sách mười hai heuristic ở chương A/01 có một câu: “thay vì hỏi đáp án là bao nhiêu, hãy hỏi với giá trị x cho trước thì đáp án có $\geq x$ không”. Chương này khai thác trọn vẹn câu đó, và lý do nó đáng có một chương riêng là vì hiệu lực bất thường của nó: rất nhiều bài tối ưu trông không có lời giải nào lại có một hàm kiểm tra hiển nhiên chỉ dài năm dòng.

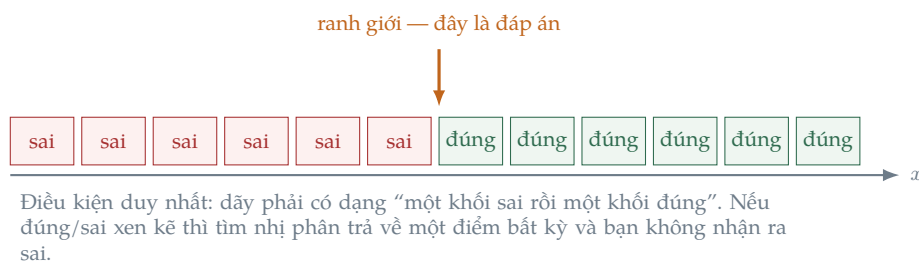
Cơ chế đằng sau khá rõ. Bài toán tối ưu buộc bạn phải *xây dựng* lời giải tốt nhất, tức là vừa phải quyết định cấu hình vừa phải bảo đảm không có cấu hình nào tốt hơn. Bài toán quyết định thì nhẹ hơn nhiều: bạn được cho sẵn ngưỡng, và chỉ cần trả lời có hoặc không — thường bằng một phép quét tham lam, vì khi ngưỡng đã cố định thì mọi lựa chọn trở nên rõ ràng. Đổi bài tối ưu thành bài quyết định là chuyển gánh nặng từ “tìm” sang “kiểm”, và kiểm thì rẻ hơn nhiều.

1.1 Khuôn chuẩn và điều kiện áp dụng

Khuôn gồm ba phần. *Một*: xác định miền giá trị của đáp án — cận dưới và cận trên chắc chắn. *Hai*: viết hàm $P(x)$ trả lời “có phương án nào đạt mức x không”. *Ba*: nhị phân trên x , giữ bất biến “ P sai với mọi giá trị nhỏ hơn lo , đúng với hi ” như ở chương A/04.

Điều kiện sống còn là *tính đơn điệu* của P : nếu đạt được mức x thì cũng đạt được mọi mức dễ hơn. Cách kiểm nhanh và đáng tin: phát biểu P thành lời và hỏi “nếu tôi làm được với ngân sách x , tôi có chắc làm được với ngân sách $x + 1$ không?”. Trong phần lớn bài đúng dạng, câu trả lời là hiển nhiên vì *chính phương án cũ vẫn hợp lệ*. Nếu bạn phải suy nghĩ lâu để trả lời thì rất có thể tính đơn điệu không có, và kỹ thuật này sẽ cho kết quả sai một cách im lặng.

Chi phí là $O(\log(\text{miền giá trị}) \times \text{chi phí kiểm tra})$. Thừa số log hầu như luôn nhỏ — với miền 10^{18} thì chỉ 60 vòng — nên bạn được phép có hàm kiểm tra khá đắt. Đây là điểm đáng nhớ: kỹ thuật này cho phép bạn “mua” một bậc log để đổi lấy một bài toán dễ hơn hẳn.



Hình 1.1. Tìm nhị phân trên đáp án không tìm trên dữ liệu mà tìm trên *miền giá trị của kết quả*. Việc bạn thật sự làm là tìm ranh giới giữa vùng “không đạt được” và vùng “đạt được”. Toàn bộ tính đúng nằm ở giả thiết rằng ranh giới đó tồn tại và chỉ có một.

1.2 Dấu hiệu nhận biết trên đề

Ba dấu hiệu, xếp theo độ tin cậy.

Dấu hiệu thứ nhất, gần như chắc chắn: đề có cụm “nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất”. Ví dụ: chia mảng thành k đoạn sao cho tổng lớn nhất của một đoạn là nhỏ nhất; đặt k trạm sao cho khoảng cách xa nhất từ một nhà tới trạm là nhỏ nhất. Dạng “cực tiểu của cực đại” hầu như luôn giải bằng kỹ thuật này, vì hàm kiểm tra “có chia được sao cho mọi đoạn $\leq x$ không” là một phép quét tham lam khi các phần tử không âm và số đoạn được giới hạn từ trên. Với số âm, phải thiết kế lại hàm kiểm tra; từ khoá minimax tự nó không chứng minh rằng kiểm tra là dễ.

Dấu hiệu thứ hai: đề hỏi giá trị tối ưu mà việc *kiểm tra* một giá trị cụ thể lại dễ. Ví dụ: tốc độ ăn nhỏ nhất để kịp thời hạn; kích thước lớn nhất của k miếng cắt bằng nhau.

Dấu hiệu thứ ba: đề hỏi “lớn nhất/nhỏ nhất sao cho một tính chất vẫn giữ được”, và tính chất đó rõ ràng dễ mất đi khi ta đòi hỏi nhiều hơn.

Có một biến thể quan trọng: *nhị phân trên tỉ số*. Với các bài tối ưu tỉ số kiểu “tìm tập con có trung bình lớn nhất”, hãy nhị phân trên giá trị tỉ số x và kiểm tra bằng cách trừ x khỏi mỗi phần tử rồi hỏi có tập con nào có tổng không âm không. Kỹ thuật này biến bài tỉ số — vốn không cộng dồn được — thành bài tổng thông thường, và nó giải cả họ bài “chu trình trung bình nhỏ nhất” hay “đoạn con có trung bình lớn nhất với độ dài tối thiểu”.

1.3 Ba cạm bẫy cài đặt

Cạm bẫy 1 — số thực và điều kiện dừng. Trên số thực, đừng lặp cho tới khi $lo = hi$ vì điều đó có thể không bao giờ xảy ra do sai số biểu diễn. Hai cách đúng: lặp một số vòng cố định — chọn k sao cho $(hi_0 - lo_0)/2^k \leq \varepsilon$ trong số học chính xác — hoặc dừng khi $hi - lo$ nhỏ hơn một ngưỡng đã chọn dựa trên yêu cầu của đề. Số vòng cần thiết phụ thuộc độ rộng ban đầu và sai số yêu cầu. Với dấu phẩy động, còn phải dừng nếu trung điểm bằng một đầu mút do làm tròn; tăng số vòng không vượt qua được độ phân giải của kiểu số.

Cạm bẫy 2 — biên và kiểu số. Miền giá trị phải chứa đáp án: cận dưới thường là 0 hoặc giá trị nhỏ nhất, cận trên phải là một giá trị chắc chắn đạt được. Với dữ liệu lớn, $(lo + hi)/2$ có thể tràn — hãy viết $lo + (hi - lo)/2$. Và kiểm tra kiểu: nếu đáp án tới 10^{18} thì cả biên lẫn giá trị trung gian trong hàm kiểm tra đều phải là 64 bit.

Cạm bẫy 3 — hàm kiểm tra sai một cách tinh vi. Đây là lỗi hay gặp nhất và khó thấy nhất, vì phần nhị phân thì đúng còn phần kiểm tra thì sai ở một trường hợp biên. Cách phòng hiệu quả nhất: viết hàm kiểm tra trước, test riêng nó với vài giá trị x cụ thể mà bạn tính được đáp án bằng tay, rồi mới ghép vào vòng nhị phân.

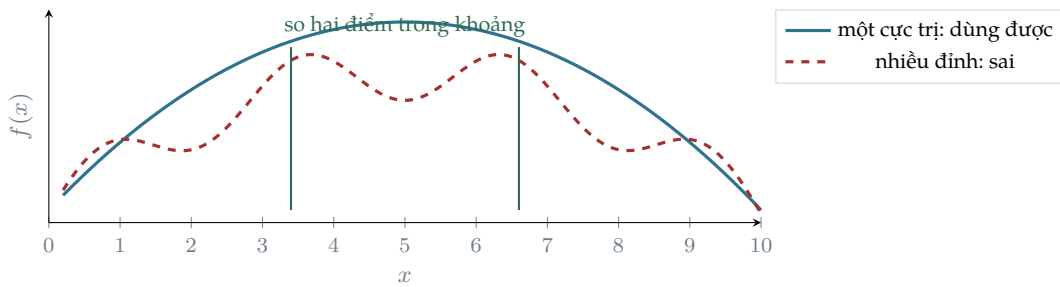
1.4 Tìm kiếm ba phân: khi hàm có một đỉnh

Nếu bài toán không cho một vị từ đơn điệu mà cho một *hàm số* tăng rồi giảm, ta không nhị phân được nhưng vẫn thu hẹp khoảng được. Ý tưởng: lấy hai điểm $m_1 < m_2$ bên trong khoảng; nếu $f(m_1) < f(m_2)$ thì cực đại không nằm bên trái m_1 , nên bỏ đoạn đầu; ngược lại bỏ đoạn cuối.

Điều kiện là hàm phải *một cực trị* — tăng rồi giảm, không có gợn nào khác. Đây là điều kiện mạnh hơn “có cực đại” rất nhiều, và đó là câu trả lời cho câu hỏi mở đầu thứ tư: nếu hàm có nhiều đỉnh địa phương thì phép so sánh hai điểm không còn nói được gì về vị trí cực đại toàn cục, và thuật toán trả về một đỉnh bất kỳ.

Trên miền số nguyên, tìm kiếm ba phân dễ sai ở những bước cuối khi khoảng còn vài phần tử; cách an toàn là thu hẹp tới khi còn dưới ba phần tử rồi thử hết. Trên số thực, dùng số vòng lặp cố định như ở mục trước.

Một biến thể đáng biết cho các bài tối ưu lồi: nếu hàm *lồi* thì có thể nhị phân trên *đạo hàm* rồi rạc $f(i+1) - f(i)$, vốn đơn điệu — và như vậy quay về bài toán nhị phân thông thường. Cách nhìn này nối chương với các tối ưu quy hoạch động ở C/16, nơi tính lồi cũng là điều kiện then chốt.



Hình 1.2. Tìm kiếm ba phân cần hàm *một cực trị*, không chỉ “có cực đại”. Với đường liền, việc so hai điểm bên trong cho biết cực đại nằm về phía nào. Với đường nét đứt nhiều gợn, cùng phép so chỉ nói lên xu hướng *cực bộ*, nên thuật toán hội tụ về một đỉnh bất kỳ.

1.5 Khi không có cấu trúc nào: tìm kiếm cực bộ

Nếu không có tính đơn điệu, không có tính lồi, và không có cấu trúc tổ hợp nào khai thác được, ta rơi vào lớp bài mà không thuật toán chính xác nào chạy nổi. Lúc đó còn một họ phương pháp: bắt đầu từ một lời giải bất kỳ và cải thiện dần bằng các thay đổi nhỏ.

Leo đồi luôn nhận thay đổi làm tốt lên; nó nhanh nhưng kẹt ở cực trị địa phương — đúng thất bại mà chương C/15 đã mô tả cho tham lam. *Mô phỏng luyện kim* thỉnh thoảng chấp nhận cả thay đổi làm tệ đi, với xác suất giảm dần theo thời gian; điều đó cho phép thoát khỏi cực trị địa phương ở giai đoạn đầu và ổn định ở giai đoạn sau. *Khởi động lại nhiều lần* thì đơn giản hơn: chạy leo đồi từ nhiều điểm xuất phát ngẫu nhiên và giữ kết quả tốt nhất.

Cần thẳng thắn về vị trí của nhóm này. Chúng không có bảo đảm chất lượng, kết quả phụ thuộc tham số và may rủi, và trong thi đấu thì hiếm khi là lời giải mong muốn. Nhưng trong công việc thật — lập lịch, bố trí, hiệu chỉnh tham số — chúng thường là thứ duy nhất chạy được trong thời gian cho phép. Nguyên tắc dùng: chỉ chuyển sang nhóm này sau khi đã chứng minh được rằng bài toán không có cấu trúc khai thác được, không phải vì lười tìm.

1.6 Một vết chạy đầy đủ: chia mảng để giảm tổng lớn nhất

Lấy $[7, 2, 5, 10, 8]$, chia thành tối đa hai đoạn không rỗng; mọi phần tử không âm. Miền đáp án là $[10, 32]$. Hàm $P(x)$ quét trái sang phải, chỉ mở đoạn mới khi thêm phần tử kế tiếp làm tổng vượt x . Quét như vậy dùng ít đoạn nhất: mỗi đoạn lấy tiền tố dài nhất còn hợp lệ; không âm bảo đảm phần đuôi không khó hơn khi ta lấy thêm phần tử vào đoạn trước.

Bảng 1.1. Tìm giá trị đúng đầu tiên, với $mid = lo + \lfloor (hi - lo)/2 \rfloor$. Đúng thì $hi = mid$, sai thì $lo = mid + 1$.

lo	hi	mid	Số đoạn	Chia tham lam tại mid
10	32	21	2: đúng	$[7, 2, 5] \mid [10, 8]$
10	21	15	3: sai	$[7, 2, 5] \mid [10] \mid [8]$
16	21	18	2: đúng	$[7, 2, 5] \mid [10, 8]$
16	18	17	3: sai	$[7, 2, 5] \mid [10] \mid [8]$

Bảng 1.1 kết thúc ở $lo = hi = 18$. Hai phép kiểm quan trọng nhất là $P(18)$ đúng và $P(17)$ sai: chúng kiểm ranh giới, không chỉ một điểm ở xa đáp án. Nếu đề yêu cầu đúng k đoạn, điều kiện “tối đa k ” tương đương khi $1 \leq k \leq n$ và các phần tử không âm: có thể tách thêm đoạn mà không tăng tổng lớn nhất.

1.7 Gốc rễ: từ giải phương trình tới ủ kim loại

Tìm kiếm nhị phân là ý tưởng cổ hơn máy tính rất nhiều: nó chính là *phương pháp chia đôi* của giải tích số để tìm nghiệm phương trình, và ý tưởng “giá trị đối dấu giữa hai đầu thì có nghiệm ở giữa” đã được dùng từ lâu trước thế kỷ 20. Nhưng phiên bản rời rạc trên mảng lại có một lịch sử đáng nhớ về sự khó cài: Knuth ghi nhận rằng thuật toán được công bố lần đầu vào năm 1946, nhưng phiên bản đầu tiên xử lý đúng *mọi* kích thước mảng chỉ xuất hiện vào năm 1962 — mười sáu năm cho một thuật toán mà ai cũng nghĩ là hiển nhiên. Cùng với quan sát của Bentley[1] rằng rất nhiều lập trình viên chuyên nghiệp viết sai nó khi phải viết từ đầu, đây là bằng chứng mạnh cho luận điểm ở chương A/04: sự đơn giản của ý tưởng không nói gì về độ khó của việc cài đúng.

Ý tưởng *tìm nhị phân trên đáp án* thì không có tác giả rõ ràng — nó là một kỹ thuật hiển nhiên với người đã quen nhìn bài toán tối ưu như bài toán quyết định. Nhưng cách nhìn đó lại có nguồn gốc lý thuyết rất rõ: trong lý thuyết độ phức tạp, mối quan hệ giữa bài toán tối ưu và bài toán quyết định là quan hệ nền tảng — người ta định nghĩa lớp NP qua bài toán quyết định chính vì lý do đó, và mọi bài tối ưu đều có một phiên bản quyết định đi kèm (D/23). Kỹ thuật của chương này là ứng dụng thực dụng của cùng ý tưởng: nếu bài quyết định dễ hơn thì hãy giải nó và tìm ranh giới.

Mô phỏng luyện kim có một gốc rễ liên ngành đẹp: năm 1983, Kirkpatrick và các cộng sự tại IBM đề xuất nó bằng cách mượn thẳng một hiện tượng vật lý. Khi làm nguội kim loại nóng chảy thật chậm, các nguyên tử có thời gian sắp xếp về cấu hình năng lượng thấp và tinh thể tạo thành đều đẹp; làm nguội nhanh thì kim loại đông cứng ở trạng thái năng lượng cao và có nhiều khuyết tật. Họ nhận ra rằng cùng cơ chế — cho phép hệ tạm thời đi lên mức năng lượng cao hơn, với xác suất giảm dần theo “nhiệt độ” — có thể dùng để thoát khỏi cực trị địa phương trong bài toán tối ưu tổ hợp. Đây là một trong những ví dụ rõ nhất về việc mượn cơ chế từ ngành khác, và nó cũng giải thích tên gọi cùng các tham số của phương pháp.

Gốc rễ: ý tưởng này ra đời từ bài toán nào

Mốc	Bài toán thúc ép lúc đó	Điều thay đổi sau đó
Trước thế kỷ 20 — phương pháp chia đôi 1946–1962 — tìm kiếm nhị phân rời rạc	Tìm nghiệm phương trình khi không có công thức đóng. Tra cứu trong bảng đã sắp.	Ý tưởng nhị phân trên miền liên tục; tổ tiên trực tiếp của cả chương này. Công bố 1946, nhưng bản đúng cho mọi kích thước mảng phải tới 1962 — bằng chứng rằng ý tưởng đơn giản không đồng nghĩa với cài đặt dễ.
Thập niên 1970 — lý thuyết độ phức tạp	Cần một cách chuẩn hoá để so độ khó của các bài toán.	Quan hệ giữa bài tối ưu và bài quyết định trở thành khái niệm nền; kỹ thuật của chương này là ứng dụng thực dụng của nó (D/23).
1983 — Kirkpatrick và cộng sự	Bài tối ưu tổ hợp lớn, leo đồi kẹt ở cực trị địa phương.	Mô phỏng luyện kim: mượn cơ chế làm nguội chậm trong vật lý để cho phép tạm thời đi ngược hướng cải thiện.

1.8 Liên hệ ngang

Liên hệ ngang: ý này còn sống ở đâu nữa

Lĩnh vực	Ý tưởng tương ứng	Giống ở đâu, khác ở đâu
Giải tích số	Phương pháp chia đôi, phương pháp Newton	Cùng ý tưởng thu hẹp khoảng chứa nghiệm. Khác: Newton hội tụ nhanh hơn nhiều nhưng cần đạo hàm và có thể phân kỳ; chia đôi chậm mà chắc — đúng đánh đổi giữa tốc độ và tính bền vững.
Gỡ lỗi phần mềm	git bisect: tìm commit đầu tiên làm hỏng	Đúng thuật toán của chương, áp lên trục thời gian của kho mã. Điều kiện áp dụng cũng giống hệt: tính đơn điệu — trước điểm đó thì tốt, sau điểm đó thì hỏng. Nếu lỗi xuất hiện rồi biến mất rồi lại xuất hiện thì công cụ trả về kết quả vô nghĩa.
Vật lý	Ủ kim loại, hạ nhiệt độ chậm để đạt trạng thái năng lượng thấp	Nguồn gốc trực tiếp của mô phỏng luyện kim, kể cả tên gọi và tham số “nhiệt độ”. Bài học chuyển được: cho phép tạm thời đi ngược hướng cải thiện là cách thoát khỏi cực trị địa phương.
Học máy	Dò tham số, dừng sớm, tìm tốc độ học	Kỹ thuật tìm tốc độ học bằng cách tăng dần và quan sát chính là tìm ranh giới trên một đại lượng gần đơn điệu. Cạm bẫy cũng giống: nếu quan hệ không đơn điệu thì kết luận sai.
Ước lượng robot, SLAM	Chọn ngưỡng loại ngoại lai, hiệu chỉnh tham số	Nhiều tham số ngưỡng có quan hệ đơn điệu với tỉ lệ chấp nhận, nên tìm nhị phân dùng được. Cạm bẫy quen thuộc: quan hệ đó chỉ đơn điệu <i>trung bình</i> , còn trên một tập dữ liệu cụ thể thì có nhiễu — và nhị phân trên hàm nhiễu cho kết quả không ổn định.
Kinh tế học, đấu giá	Tìm giá cân bằng bằng cách dò tăng dần	Cùng cấu trúc: cầu giảm khi giá tăng — một vị từ đơn điệu — nên giá cân bằng tìm được bằng chia đôi. Đây là cơ chế của nhiều thuật toán đấu giá thật.
Tối ưu hoá	Tìm kiếm cục bộ, thuật toán tiến hoá	Cùng họ với mục 5. Nhận xét đáng nhớ: các phương pháp này không có bảo đảm, nên chúng chỉ nên được dùng <i>sau khi</i> đã xác nhận bài toán không có cấu trúc khai thác được.

TRẢ LỜI CÂU HỎI MỞ ĐẦU

- A1. Vì bài tối ưu đòi bạn vừa xây dựng cấu hình vừa bảo đảm không có cấu hình nào tốt hơn, còn bài quyết định cho sẵn ngưỡng nên mọi lựa chọn trở nên rõ ràng. Khi đã biết “mỗi đoạn không được vượt x ”, việc chia mảng thành ít đoạn nhất trở thành một phép quét tham lam hiển nhiên: cứ nhồi vào đoạn hiện tại tới khi vượt thì mở đoạn mới. Nói cách khác, ngưỡng cố định đã loại bỏ phần cần cân nhắc, và cái còn lại thường chỉ là một lượt duyệt.
- A2. Kiểm bằng cách phát biểu vị từ thành lời rồi hỏi “nếu làm được với ngân sách x thì có chắc làm được với $x + 1$ không”. Trong các bài đúng dạng, câu trả lời hiển nhiên vì chính phương án cũ vẫn hợp lệ. Nếu nhầm — vị từ không đơn điệu — thì tìm nhị phân vẫn chạy và vẫn trả về một số, nhưng số đó là một điểm chuyển bất kỳ chứ không phải đáp án; đây là lỗi im lặng, và chỉ stress test với vết cạn mới bắt được.
- A3. Trên số thực, phép chia đôi có thể không bao giờ làm hai đầu bằng nhau do sai số biểu diễn, nên vòng lặp treo. Hai cách đúng: lặp một số vòng cố định (100 vòng là quá đủ, vì mỗi vòng giảm khoảng đi một nửa) hoặc dừng khi $hi - lo$ nhỏ hơn một ngưỡng chọn theo yêu cầu độ chính xác của đề. Cách lặp cố định an toàn hơn vì độ chính xác tương đối không phụ thuộc độ lớn của giá trị.
- A4. Vì phép so sánh hai điểm bên trong chỉ cho thông tin về vị trí cực đại nếu hàm không có gợn nào khác. Với hàm nhiều đỉnh, việc $f(m_1) < f(m_2)$ không loại trừ được khả năng cực đại toàn cục nằm bên trái m_1 — nó chỉ nói rằng *quanh đó* hàm đang đi lên. Kết quả là thuật toán hội tụ về một đỉnh địa phương nào đó. “Có cực đại” là điều kiện quá yếu; phải là tăng rồi giảm, không có ngoại lệ.
- A5. Còn nhóm tìm kiếm cục bộ: leo đồi, mô phỏng luyện kim, khởi động lại nhiều lần, các thuật toán tiến hoá. Chúng không có bảo đảm chất lượng và kết quả phụ thuộc tham số, nên chỉ nên dùng sau khi đã xác nhận không còn cấu trúc nào khai thác được. Trong công việc thật — lập

lịch, bố trí, hiệu chỉnh — chúng thường là thứ duy nhất chạy được trong thời gian cho phép, và lúc đó việc cần làm là đo phân phối kết quả thay vì tin vào một lần chạy.

NÓI LẠI THẬT ĐƠN GIẢN

Có một mẹo rất đơn giản mà lại giải được rất nhiều bài khó: thay vì hỏi “đáp án tốt nhất là bao nhiêu”, hãy hỏi “liệu có đạt được mức này không”.

Ví dụ: bạn cần chia một dãy số thành phần liên tiếp sao cho phần nặng nhất càng nhẹ càng tốt. Nghĩ trực tiếp thì rối. Nhưng nếu tôi hỏi “có chia được sao cho mọi phần đều không quá 100 không?” thì bạn trả lời được ngay: cứ nhồi số vào phần hiện tại tới khi sắp vượt 100 thì mở phần mới, cuối cùng đếm xem có quá năm phần không. Chỉ vài dòng.

Bây giờ để ý điều này: nếu chia được với ngưỡng 100 thì chắc chắn cũng chia được với ngưỡng 120 — ngưỡng rộng hơn thì chỉ dễ hơn. Nghĩa là câu trả lời “được / không được” xếp thành một dãy: không, không, không, rồi được, được, được. Đáp án chính là chỗ chuyển từ “không” sang “được”, và tìm chỗ chuyển đó thì đúng là trò đoán số: hỏi ở giữa, rồi thu hẹp một nửa.

Đó là toàn bộ kỹ thuật. Dấu hiệu để nhận ra nó trên đề bài rất dễ nhớ: khi thấy chữ “nhỏ nhất của cái lớn nhất” hoặc “lớn nhất của cái nhỏ nhất”, gần như chắc chắn là nó.

Có đúng một điều phải kiểm tra, và nếu bỏ qua thì chương trình vẫn chạy nhưng ra số sai: dãy trả lời phải thật sự có dạng “một khối không rồi một khối có”. Nếu “được” và “không được” xen kẽ nhau thì trò đoán số vô nghĩa, vì bạn sẽ dừng lại ở một chỗ chuyển ngẫu nhiên nào đó.

Một câu: Đổi câu hỏi “bao nhiêu” thành câu hỏi “có đạt được mức này không”, rồi đoán số trên mức đó.

Chỗ không rút gọn được: Việc chứng minh vị từ đơn điệu vẫn phải làm riêng cho từng bài; không có dấu hiệu bề ngoài nào bảo đảm điều đó.

Active recall (che cột phải, tự trả lời, rồi mở ra đối chiếu)

Thuật ngữ / câu hỏi	Trả lời
••• Ý tưởng cốt lõi	Đổi bài tối ưu thành bài quyết định: “có đạt được mức x không”; ngưỡng cố định làm mọi lựa chọn trở nên rõ ràng, hàm kiểm tra thường chỉ là một lượt quét tham lam.
••• Điều kiện sống còn	Vị từ phải đơn điệu: đạt được x thì đạt được mọi mức dễ hơn. Kiểm bằng câu “phương án cũ có còn hợp lệ với ngân sách rộng hơn không”.
••• Dấu hiệu trên đề	“Nhỏ nhất của giá trị lớn nhất” hoặc “lớn nhất của giá trị nhỏ nhất” — gần như chắc chắn là kỹ thuật này.
••• Chi phí	$O(\log(\text{miền giá trị}) \times \text{chi phí kiểm tra})$; log của 10^{18} chỉ là 60, nên hàm kiểm tra được phép khá đắt.
••• Ba cạm bẫy cài đặt	Số thực: lập số vòng cố định thay vì chờ $lo = hi$. Biên và tràn: dùng $lo + (hi - lo)/2$, kiểu 64 bit. Hàm kiểm tra sai tinh vi: test riêng nó trước khi ghép.
•• Nhị phân trên tỉ số	Bài “trung bình lớn nhất”: nhị phân trên x , trừ x khỏi mỗi phần tử, hỏi có tổng không âm không — biến bài tỉ số thành bài tổng.
•• Tìm kiếm ba phần	Cho hàm <i>một cực trị</i> ; so hai điểm trong khoảng để loại một đầu. Nhiều đỉnh địa phương thì sai.
•• Hàm lồi thì nhị phân được	Nhị phân trên đạo hàm rời rạc $f(i+1) - f(i)$, vốn đơn điệu — quay về bài nhị phân thường.
•• Tìm kiếm cục bộ	Leo đồi (kết cực trị địa phương); mô phỏng luyện kim (chấp nhận bước xấu với xác suất giảm dần); khởi động lại nhiều lần.
•• Khi nào dùng tìm kiếm cục bộ	Chỉ sau khi xác nhận bài toán không có cấu trúc khai thác được; không có bảo đảm chất lượng nên phải đo phân phối kết quả.
• Liên hệ với git bisect	Cùng thuật toán trên trục thời gian của kho mã; cùng yêu cầu đơn điệu — lỗi phải xuất hiện từ một điểm và không biến mất.

I Câu hỏi ôn tập có đáp án (che phần đáp án, tự trả lời trước)

1. Nhị phân trên đáp án và nhị phân trên mảng khác nhau thế nào?

| Khác ở *cái được tìm trên*. Nhị phân trên mảng tìm một phần tử trong dữ liệu đã sắp; nhị phân trên đáp án tìm một giá trị trong *miền kết quả*, và dữ liệu có thể hoàn toàn không sắp. Điểm chung là cả hai đều cần một vị từ đơn điệu, và cả hai đều dùng cùng khuôn bất biến ở A/04. Nhận ra chúng là một giúp bạn viết cả hai bằng cùng một mẫu code và ít sai hơn.

2. Tôi nên nhị phân trên số nguyên hay số thực khi đáp án là số thực?

| Nếu đề cho phép sai số, hãy xem có thể nhân lên thành số nguyên không — ví dụ đáp án luôn là bội của 0,001 thì nhân 1000 và làm việc trên số nguyên, tránh hẳn mọi vấn đề sai số. Nếu không, dùng số thực với số vòng lặp cố định. Một mẹo phụ: nhiều bài trông có đáp án thực nhưng đáp án thật sự luôn nằm trong một tập hữu hạn giá trị (một hiệu, một tỉ số của các số nguyên trong dữ liệu) — lúc đó nhị phân trên tập đó cho kết quả chính xác tuyệt đối.

3. Hàm kiểm tra của tôi tốn $O(n \log n)$. Tổng $O(n \log n \log C)$ có quá chậm không?

| Thường là không, và đây là lý do kỹ thuật này mạnh. Với $n = 10^5$ và $C = 10^9$: $10^5 \times 17 \times 30 \approx 5 \cdot 10^7$ — nằm trong ngân sách. Điều cần kiểm là hằng số của hàm kiểm tra: nếu nó cấp phát bộ nhớ hoặc sắp xếp lại từ đầu mỗi lần thì hãy chuyển việc sắp xếp ra *ngoài* vòng nhị phân, vì dữ liệu không đổi giữa các lần kiểm.

4. Làm sao nhận ra tính đơn điệu khi nó không hiển nhiên?

| Ba cách. Một, viết vị từ ra thành lời và tìm một lập luận kiểu “phương án hợp lệ cho x vẫn hợp lệ cho $x + 1$ ”. Hai, tính tay giá trị vị từ cho vài x liên tiếp trên một ví dụ nhỏ và nhìn dãy kết quả. Ba, viết bộ sinh test nhỏ, tính vị từ cho mọi x trong miền, và kiểm bằng máy rằng dãy có dạng “khởi sai rồi khỏi đúng”. Cách ba tốn năm phút và cho câu trả lời chắc chắn — đáng làm khi nghi ngờ.

5. Bài của tôi có hai đại lượng cần tối ưu cùng lúc. Kỹ thuật này còn dùng được không?

| Có, theo hai cách. Nếu hai đại lượng có thứ tự ưu tiên — tối ưu cái thứ nhất trước, cái thứ hai chỉ để phá hoà — thì nhị phân trên cái thứ nhất, rồi trong hàm kiểm tra hãy trả về giá trị tốt nhất của cái thứ hai. Nếu bài yêu cầu tối ưu một tổ hợp tuyến tính có tham số, đó là bài toán tìm kiếm tham số và thường giải bằng nhị phân trên tham số cộng một tính chất lồi — cùng ý tưởng với nhị phân trên tỉ số ở mục 2.

6. Mô phỏng luyện kim có đáng học cho thi đấu không?

| Đáng biết là có, nhưng hiếm khi cần. Nó xuất hiện trong một số kỳ thi cho phép lời giải xấp xỉ và chấm theo điểm chất lượng, và trong các bài tối ưu hình học nhỏ. Trong các kỳ thi chấm đúng/sai thì hầu như không dùng. Ngược lại, trong công việc thật thì nó rất hữu ích — lập lịch, bố trí, hiệu chỉnh tham số — nên nó thuộc loại “học một lần cho biết cách dùng, không cần luyện nhiều”.

7. Có bài nào nhìn giống nhị phân trên đáp án mà thực ra không phải không?

| Có, và dấu hiệu là vị từ không đơn điệu. Ví dụ: “tìm giá trị x sao cho số cách đạt đúng x là lớn nhất” — vị từ ở đây không phải dạng “đạt được hay không” mà là một hàm đếm, và nó có thể lên xuống tùy ý. Một ví dụ khác: các bài mà tăng ngân sách lại làm phương án cũ *không còn hợp lệ* vì có ràng buộc kiểu “phải dùng hết”. Khi thấy chữ “đúng bằng” hoặc “dùng hết” trong đề, hãy dừng lại kiểm tính đơn điệu cẩn thận.

I Bài tập (mức: ● khởi động ●● chuẩn ●●● khó)

Bài	Nguồn	Điều cần nhận ra
● Tìm căn bậc hai nguyên của một số	Bài kinh điển	Nhị phân trên đáp án; chú ý tràn khi bình phương.
● Tốc độ nhỏ nhất để hoàn thành trong h giờ	LeetCode “Koko Eating Bananas”	Bài mẫu nhập môn; viết vị từ ra thành lời trước.

Bài	Nguồn	Điều cần nhận ra (tiếp)
•• Chia mảng thành k đoạn sao cho tổng lớn nhất nhỏ nhất	CSES “Array Division”[2]	Dạng “cực tiểu của cực đại” kinh điển; hàm kiểm tra là một lượt tham lam.
•• Đặt k trạm sao cho khoảng cách xa nhất nhỏ nhất	Bài kinh điển	Cùng dạng; chú ý biên của miền giá trị.
•• Số nhỏ nhất x sao cho có ít nhất k bội số của các số cho trước không vượt x	CSES “Factory Machines”[2]	Nhị phân trên thời gian; cẩn thận tràn khi cộng số lượng.
•• Chuỗi con lặp lại dài nhất	Bài kinh điển	Nhị phân trên độ dài cộng bấm chuỗi ($B/12$); tính đơn điệu ở đây rất rõ.
••• Đoạn con có trung bình lớn nhất với độ dài tối thiểu k	Bài kinh điển	Nhị phân trên tỉ số: trừ x khỏi mỗi phần tử rồi hỏi có đoạn tổng không âm không.
••• Chu trình có trung bình trọng số nhỏ nhất	Bài kinh điển	Cùng mẹo tỉ số, kết hợp phát hiện chu trình âm bằng Bellman–Ford ($C/17$).
••• Tìm điểm cực tiểu của một hàm một cực trị trên số thực	Bài kinh điển	Tìm kiếm ba phân với số vòng cố định; so với cách nhị phân trên đạo hàm rời rạc.
••• Bài toán tối ưu nhỏ giải bằng mô phỏng luyện kim, so với vét cạn	Bài tập thực nghiệm	Chạy 100 lần và vẽ phân phối kết quả — mục tiêu là thấy tận mắt “không có bảo đảm” nghĩa là gì.
••• Kiểm tra tính đơn điệu của một vị từ bằng máy trên dữ liệu nhỏ	Bài tập kiểm thử	Viết vòng lặp tính vị từ cho mọi x và khẳng định đây có dạng “sai rồi đúng”; công cụ này nên có sẵn trong thư viện của bạn.

Kết chương và đọc tiếp

Kỹ thuật của chương này rẻ tới mức nên là một trong những thứ đầu tiên bạn thử khi gặp bài tối ưu: đọc đề, tìm cụm “nhỏ nhất của lớn nhất”, và hỏi “nếu tôi cố định ngưỡng thì việc kiểm tra có dễ không”. Hai điều đáng mang theo: tính đơn điệu là điều kiện duy nhất nhưng bắt buộc, và nó phải được kiểm chứ không được giả định; và log là cái giá rất rẻ để đổi lấy một bài toán dễ hơn hẳn.

Chương cuối của phần C là công cụ cắt ngang thứ hai, và nó đã xuất hiện thấp thoáng ở bốn chương trước: bấm phổ dụng, treap, chốt quicksort, ước lượng cây quay lui. Chương C/20 phát biểu nguyên lý chung đằng sau cả bốn — *khi trường hợp xấu phụ thuộc vào dữ liệu, hãy chuyển sự phụ thuộc sang một nguồn ngẫu nhiên mà bạn kiểm soát* — và chỉ ra rằng ngẫu nhiên hoá không phải sự cầu may mà là một công cụ thiết kế có bảo đảm.

Nguồn

- [1] J. Bentley, *Programming Pearls*, 2nd ed. Addison-Wesley, 2000.
- [2] A. Laaksonen. “CSES Problem Set.” (2026), address: <https://cses.fi/problemset/> (visited on 09/06/2026).